

OOP Mini Project Final Report

CPU Scheduling Algorithms Simulation

Group : 20
Class code : **162060**
Member list :

No	Name	Student ID
1	Hoàng Trung Dũng	20235488
2	Nguyễn Tất Hưng	20235500
3	Nguyễn Ngọc Tuấn Anh	202416658

I. Project overview

This project is for the Object Oriented Programming class in the 20251 semester. It is a program that simulates CPU scheduling algorithms given some processes that need to be executed. We were to implement three scheduling algorithms and a GUI that displays how those algorithms run on a Gantt chart. We completed all the functional features on time and this is our report for it.

There are five sections in this report: The introduction (the current section), requirement specification, program design, implementation, and testing and verification.

The next part is a formal reiteration of the requirement specification.

II. Requirement specification

Here is a formal reiteration of what we have to do:

We need to make a program that visualizes FCFS, SJN, and RR. The GUI needs 4 screens. The Main screen needs to have a dropdown or something for the user to choose among the aforementioned scheduling algorithms. The Main screen also has a Help button that creates a screen that explains the algorithm, and a Quit button that creates a screen that asks for confirmation before closing the program. Next, we also need to have a Simulation screen that displays all the processes that have to be executed. The Simulation screen also has a Gantt chart, and statistics such as average waiting time, turnaround time, and CPU utilization.

So in our project, we actually bundled the Main screen and Simulation screen together. The Simulation screen is built into the Main screen. This is because as per the specification, the Main screen contains the algorithm selection, which belongs to the simulation process, and thus the Simulation screen.

Here is the use-case diagram for our program based on the :

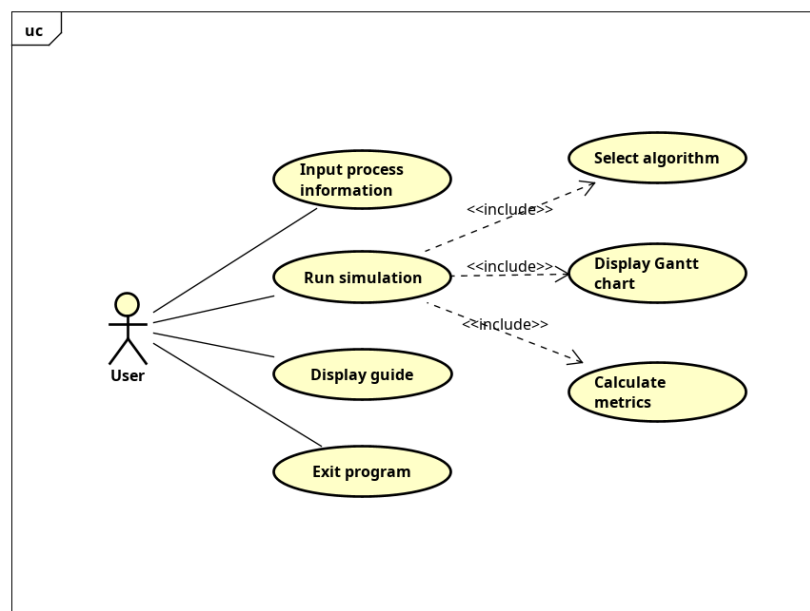


Figure 1. The use-case diagram based on initial requirements

As you can see there are 4 main use cases for the program: Input process information (the program will have to record user input for process information), run simulation, display guide, and exit program. And the run simulation use case includes processing user selection

of one of three scheduling algorithms, displaying the Gantt chart and calculating metrics. These three things must be done every time the user clicks the “Run simulation” button.

Our program will have nine main features:

1. Receive user input containing process information
2. Allow the user to choose among one of three scheduling algorithms
3. Run the FCFS scheduling algorithm
4. Run the SJN scheduling algorithm
5. Run the RR scheduling algorithm
6. Render a Gantt chart based on the results of the chosen algorithm
7. Offer a Help menu that explains how each algorithm works
8. Offer an Exit button that spawns a pop-up to ask for confirmation before terminating the program
9. A button to clear current processes and reset the GUI

It should be noted that the 9th feature was not at all mentioned in the problem statement and is in fact not an official functional requirement. This feature was added because we deem it an important aspect of a good user experience: Without it, the user would have to exit the program and restart it every time they want to run the simulation on a different set of processes.

III. Program design

Before we get to the actual programming of the simulation software. We need a design. First of all, it is helpful to look at a simple, somewhat tentative design of the program:

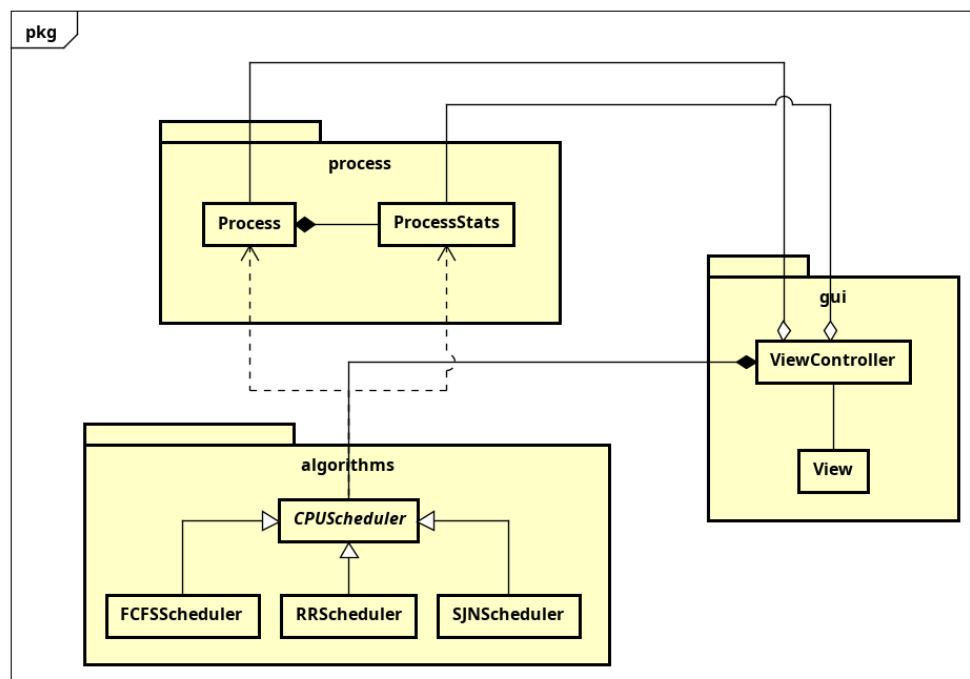


Figure 2. The simple version of the program design, class diagram where all the classes do not have the attributes or methods field yet.

In this design, there are 3 main packages. The first package contains the Process and ProcessStats classes. The Process class is an entity and represents each individual class that might be inputted into the program. The ProcessStats class is needed because each process might have different runtimes and wait times depending on different scheduling algorithms. Each Process can have multiple ProcessStats.

Next is the algorithms package, which contains an abstract class CPUScheduler that the concrete classes FCFSScheduler, RRScheduler, and SJNScheduler inherit from. This is a rather standard design decision that adheres to the Abstraction principle of OOP. Nothing more to comment here.

Finally is the gui package which contains the View and ViewController. Note that the “Model” component of our MVC model is actually split into the two aforementioned packages, process and algorithms. Anyway, the ViewController will initiate CPUScheduler as well as contain Process and ProcessStats. This is why there is a composition relationship between ViewController and CPUScheduler whereas the relationship between it and Process and ProcessStats is aggregation.

With the simplified design out of the way, we shall now work out a more detailed design to be implemented in code:

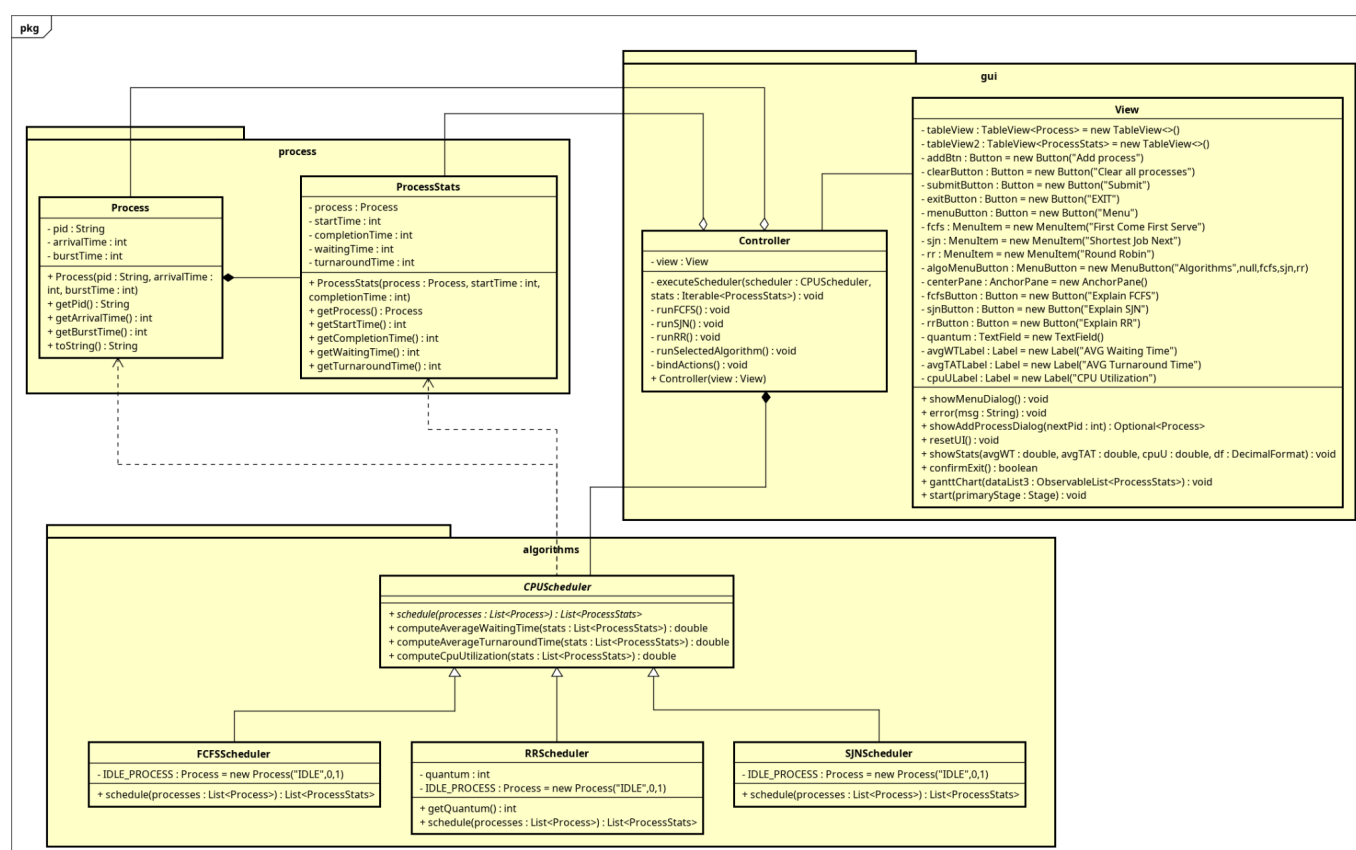


Figure 3. The detailed class diagram for the program design.

As can be seen, the Process class contains the attributes typical of any program in an operating system like process ID, arrival time and burst time. There are also getters for these attributes. ProcessStats contain attributes on completion time, waiting time and turnaround time, which vary depending on the chosen scheduling algorithm.

Next, the abstract class CPUScheduler has a single abstract method schedule(process) that each child class has to implement by itself. But it also has three concrete classes that compute the average waiting time, average turnaround time and CPU utilization. The RRScheduler class has an additional field compared to the other two classes because round robin also needs a quantum time to be set.

The controller will have a function to execute the scheduling algorithms based on the given process information. It also has a `bindAction()` function to bind all of the buttons and toggles in View with code that does actual work. The View class looks the longest and most complicated but it's just setting up all of the UI elements such as all of the buttons to submit new processes and spawn a help window or spawn an exit window, and all of the labels for the scheduling algorithm statistics. Note that the View should also have a function called `resetUI()` to clear the GUI when all the processes are removed to run another set of processes.

With this detailed program design, the actual code implementation can happen and is documented in the section below.

IV. Implementation

As always, the implementation does somewhat stray from the original design in that it becomes more complicated. There have been added classes and relationships in the final program implementation, but the general structure has stayed largely the same.

The implementation class diagram, which is generated by astah using the real code that we wrote, is too sizable for the confinements of an A4 page, so it currently resides in our Github repository, in the design folder.

Although it looks more complicated, and there are more fields and methods for some of the classes that are present in Figure 3, the integrity and structure of the program should be more or less intact and unchanged. The GUI elements are designed with JavaFX and it looks very intuitive to use. Here is an image of the Main screen along with the Simulation screen embedded in it:

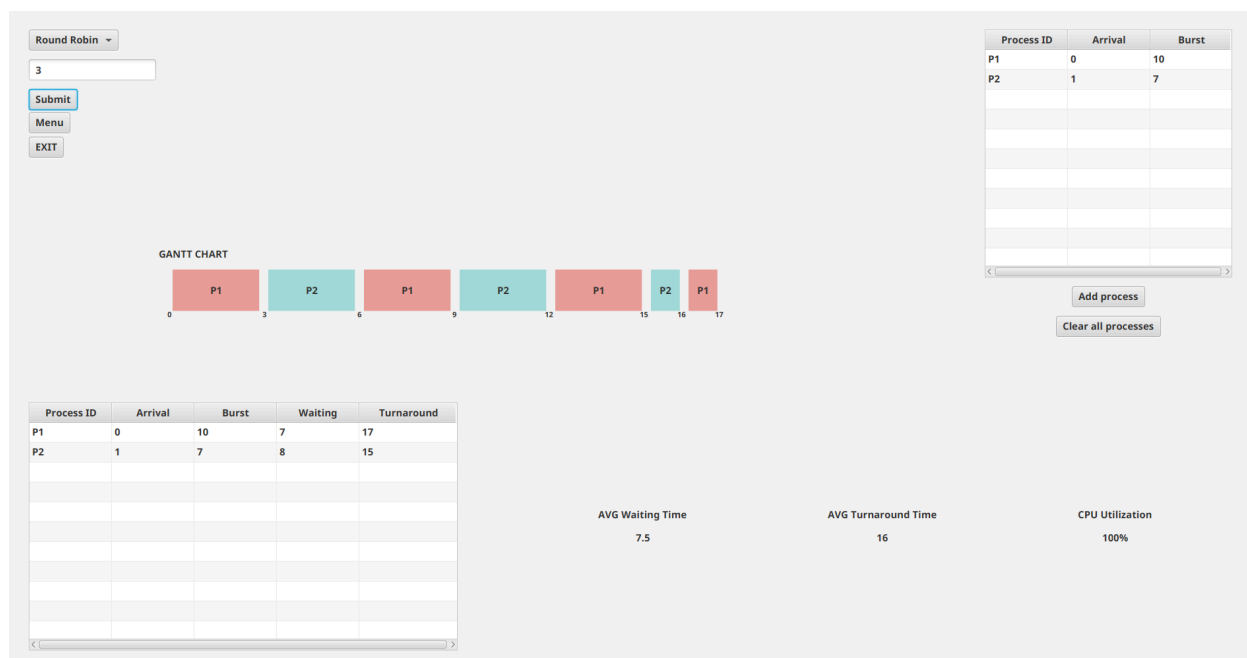


Figure 4. An image of the implementation of the program running the round robin algorithm with two processes and a time quantum of 3.

The next section contains a more in-depth testing and verification of all the functional features mentioned in section II.

V. Testing and verification

The GUI aside, we have successfully implemented all eight of the required functional features, along with one added feature that was deemed important by us.

Test number	001
Title	Input processes
Description	Verify that the program correctly receives inputted process data
Current status	Passed
Tester	Hoàng Trung Dũng
Date completed	14/1/2026

Test number	002
Title	Choose algorithm
Description	Verify that the program allows the user to choose one of many offered algorithms
Current status	Passed
Tester	Hoàng Trung Dũng
Date completed	14/1/2026

Test number	003
Title	FCFS runs correctly
Description	Verify that the FCFS algorithm option runs and returns correct statistics (average waiting time, turnaround time, and CPU utilization)
Current status	Passed
Tester	Nguyễn Ngọc Tuấn Anh
Date completed	14/1/2026

Test number	004
Title	SJN runs correctly
Description	Verify that the SJN algorithm option runs and returns correct statistics (average waiting time, turnaround time, and CPU utilization)
Current status	Passed
Tester	Nguyễn Tất Hưng
Date completed	14/1/2026

Test number	005
Title	RR runs correctly
Description	Verify that the RR algorithm option runs and returns correct statistics (average waiting time, turnaround time, and CPU utilization)
Current status	Passed
Tester	Nguyễn Tất Hưng
Date completed	14/1/2026

Test number	006
Title	Gantt chart rendering
Description	Verify that the program correctly renders Gantt charts given input data
Current status	Passed
Tester	Hoàng Trung Dũng
Date completed	14/1/2026

Test number	007
Title	Help menu
Description	Verify that the help button works and the help menu is displayed
Current status	Passed
Tester	Hoàng Trung Dũng
Date completed	14/1/2026

Test number	008
Title	Exit button
Description	Verify that the exit button works and the program exits correctly when the user confirms exit
Current status	Passed
Tester	Hoàng Trung Dũng
Date completed	14/1/2026

Test number	009
Title	Clear process button
Description	Verify that the clear process button correctly removes all process data lists and resets the GUI
Current status	Passed
Tester	Hoàng Trung Dũng
Date completed	14/1/2026

A working, live demo has been recorded and uploaded to the drive of one of our members and the link to which is present in the Google sheet.